# OPTIMIZED INVENTORY MANAGEMENT USING A HYBRID MACHINE LEARNING MIXED-INTEGER LINEAR PROGRAMMING APPROACH.

**Hailley Wanjiru Njunji**

**Supervised by: Dr. Dennis Kaburu**

*A proposal submitted to the Department of Information Technology in the School of Computing and Information Technology in partial fulfilment of the requirement for the award of the degree of Bachelor of Science in Information Technology in the Jomo Kenyatta University of Agriculture and Technology.*

*May, 2026*

**Declaration**

This proposal/research project is my original work and has not been presented for a degree in any other University

……. ……

Signature Date

Hailley Wanjiru Njunji

This proposal/research project has been submitted for examination with my approval as University Supervisor

…… …….

Signature Date

Dr. Dennis Kaburu

**Abstract**

In many small to medium organizations inventory management is characterized by reactive, manual heuristics and ad hoc decision making. These practices often culminate in a significant end of month operational crisis, where excessive labour is required to reconcile stock levels and rectify procurement errors. Though functional, these methods leave room for risks like overstocking and stock outs which lead to lost revenue and service level failures. This research aims to mitigate existing inefficiencies by designing and assessing an integrated framework that combines Machine Learning (ML) with Mixed-Integer Linear Programming (MILP) for inventory optimization. Guided by the CRISP-DM methodology, the study employs an XGBoost regressor to capture fragmented demand patterns, generating stochastic forecasts that subsequently feed into an MILP model constructed using the PuLP library.

The proposed framework mathematically models inventory flow while rigorously incorporating constraints such as storage capacity, holding costs and fixed ordering charges. Using controlled discrete event simulations, the hybrid ML–MILP policy is evaluated against a conventional fixed interval manual approach. Anticipated results suggest that this mathematically structured method can substantially lower overall inventory costs, stabilize stock availability and reduce the need for labour intensive reconciliation processes. In doing so, the study offers a data driven alternative to heuristic inventory practices, thereby promoting greater operational efficiency and enhancing data reliability within SME packaging enterprises.

## Table of Contents

## List of tables

## List of figures

## Acronyms

**LP -** Linear Programming

**MILP -** Mixed-Integer Linear Programming

**ML –** Machine Learning

**ISO -** International Organization for Standardization

**SME -** Small and Medium Enterprise

**POS –** Point of Sale

## Definition of Terms

**Inventory Policy –** This is a strategic framework used by companies to manage and control inventories of physical products, encompassing decisions related to setup, production, holding, ordering and shortage costs.

**Stock-out -** A situation in which there are no goods of a particular kind available for sale.

**Holding cost -** The cost of storing goods or owning assets.

**Lead time -** The time between ordering a product and receiving it.

**Predictive analysis –** This is the practice of using statistical algorithms and machine learning techniques to analyse historical data, identify patterns, and predict future outcomes.

**Prescriptive analysis –** This is the area of Business Analytics dedicated to searching out the best solution for day-to-day occurring problems.

**Bulk to Break –** This is the process of receiving large volume shipments of a single commodity and dividing them into smaller shipments or units of measure to be delivered to various customers or used in further production.

**Unit of Measure –** This is a definite magnitude of a quantity, defined and adopted by convention or by law, that is used as a standard for measurement of the same kind of quantity

**Manual** - refers to relying primarily on human judgment and fixed procedural rules rather than mathematically optimized decision models.

**Point of Sale** - the combination of hardware and software that businesses use to process customer transactions, track inventory, and manage sales data

# CHAPTER 1: INTRODUCTION

## 1.1. Background

Inventory is essential for any business's operation therefore proper inventory management is crucial for smooth running of day to day activities. Poor inventory management often leads to overstocking, shortages, tied-up working capital and increased operational demands. To mitigate these issues, manufacturing and distribution organizations worldwide are turning to data driven inventory control strategies that enable consistent and cost effective decision making. In particular, the food, retail and packaging supply industries are moving toward lean and evidence-based inventory practices that aim to minimize waste while maintaining service levels (Chopra & Meindl 2019).

The global landscape of operations research has been reshaped by the integration of predictive and prescriptive analytics. Contemporary inventory systems now harness Machine Learning (ML) to move beyond reliance on historical averages, applying supervised learning techniques to uncover intricate, non-linear demand behaviours and seasonal fluctuations (Zhang & Huang, 2020; Kumar & Goyal, 2025). These advanced forecasts then serve as essential inputs for prescriptive optimization approaches, including Linear Programming (LP) and Mixed-Integer Linear Programming (MILP) (Yousefi & Lari, 2022). In more developed economies, linear programming (LP) and mixed-integer linear programming (MILP) are frequently used to assist prescriptive inventory choices, enabling businesses to choose the best ordering schedules and replenishment quantities (Pinto & Ferreira, 2021; Vicente, 2025). These methods replace reactive, judgment based inventory management techniques with organized, statistically based decision frameworks (Nguyen & Shiraki, 2021; Munyoka & Doyo, 2022).

The client's challenge originates at the crossroads of demand volatility and the absence of data driven decision support mechanisms. Although the organization's point of sale (POS) system effectively standardizes transactional data, including unit of measure handling, this capability is not extended to higher level analytical processes. In the absence of predictive tools such as Machine Learning to forecast fragmented demand, and prescriptive models like MILP to or optimizing replenishment decisions, the organization remains locked in a reactive mode of operation. This cycle culminates in a persistent reconciliation bottleneck, where staff are compelled to work long hours at the end of the month conducting manual stock counts and adjustments. The motivation for this research is the pressing need to mitigate these labour intensive inefficiencies by designing a hybrid ML–MILP framework. By establishing a unified source of truth through integrated forecasting and optimized conversion, the study seeks to eliminate the root causes of the end of month crisis and align the client's processes with contemporary, lean and mathematically robust practices.

## 1.2.Project Overview

This project is within the research area of prescriptive analytics and inventory optimization, emphasizing the integration of predictive modelling with mathematical programming to support evidence based replenishment strategies. Across the globe, operations research has advanced to help organizations navigate increasingly complex supply chains by leveraging computational methods that account for multi-dimensional constraints such as fluctuating demand, budgetary restrictions and capacity limits (Chopra et al., 2019; Zhang & Huang,

2020). This research departs from static inventory models by adopting a dual-stage Forecast then Optimize pipeline, a paradigm in which predictive outputs directly shape the parameters and constraints of optimization frameworks (Sani et al., 2021; Yousefi & Lari, 2022).

Machine learning techniques have been extensively applied to demand forecasting, leveraging historical sales data to uncover complex patterns and temporal dependencies (Zhang & Huang, 2020; Kumar & Goyal, 2025). While ML models excel at prediction, they do not inherently determine optimal replenishment decisions or account for rigid operational constraints such as storage capacity and fixed ordering costs (Bertsimas & Kallus, 2018; Sani et al., 2021). In the packaging sector, where sales are fragmented and highly variable, forecasts alone cannot resolve the fundamental procurement challenge (Nguyen & Shiraki, 2021; Munyoka & Doyo, 2022). Accordingly, this research employs ML as a predictive layer to generate high-fidelity demand forecasts, capturing seasonal fluctuations and latent trends that inform subsequent decision making (Yousefi & Lari, 2022; Namazian et al., 2025).

To overcome the limitations of purely predictive models, prescriptive analytics, particularly Mixed-Integer Linear Programming (MILP), is utilized to translate predictive insights into actionable inventory decisions (Sani et al., 2021). MILP enables the simultaneous representation of discrete choices e.g., whether to place an order and continuous variables e.g., order quantities within a unified optimization framework (Pinto & Ferreira, 2021; Vicente, 2025). This mathematical rigor is critical for inventory systems that incur fixed delivery costs and must operate under capacity constraints (Zhang & Huang, 2020; Yousefi & Lari, 2022). Within this study, the MILP model functions as the computational engine that determines optimal replenishment quantities and timing based on forecasted demand generated from the machine learning stage. In doing so, procurement is precisely aligned with anticipated production needs, ensuring both efficiency and accuracy in inventory management.

At the local level, packaging material suppliers face difficulties such as demand variability, reliance on manual inventory tracking and time consuming reconciliation tasks. To tackle these challenges this research seeks to establish a reliable, automated decision support framework. The proposed approach is intended not only to stabilize inventory levels but also to illustrate how advanced computational methods can be scaled to strengthen the resilience and efficiency of SME level supply chain operations. In doing so, it aims to reduce dependence on manual, error prone stock keeping practices and promote more consistent, data driven inventory management.

### 1.3.Statement of the Problem

Despite the critical role of inventory in operational efficiency, many packaging material suppliers continue to manage stock using manual checks, fixed interval ordering and subjective judgment. These heuristic based methods frequently overlook non-linear demand fluctuations and purchase lead times (Nguyen & Shiraki, 2021; Munyoka & Doyo, 2022). Although modern point of sale (POS) systems have improved transactional accuracy and standardized inventory records, these systems primarily support operational activities and do not provide analytical capabilities for forecasting or optimization. As a result, inventory decisions remain largely reactive and experience driven.

A major consequence of these practices is the recurring end of month inventory reconciliation procedure, which requires workers to put in extra time to confirm stock quantities, update records and make accurate purchase decisions. This not only raises the risk of human error but also interferes with regular operations and increases labour cost. Moreover, reactive inventory decisions also raise the risk of stock-outs, which result in lower service levels and lost sales or overstocking, which results in needless storage expenses.

A significant shortcoming in existing inventory management procedure is the absence of mathematically grounded decision support framework that integrates predictive and prescriptive intelligence. This deficiency creates a predictive gap in anticipating demand variability and a prescriptive gap in determining optimal replenishment decisions under operational constraints (Nguyen & Shiraki, 2021). There is limited empirical evaluation of how optimization based inventory policies can reduce operational waste and stabilize inventory levels in packaging material supply environments (Munyoka & Doyo, 2022). This study therefore seeks to address this challenge by evaluating whether a hybrid framework that combines Machine Learning (ML) for demand forecasting with Mixed-Integer Linear Programming (MILP) for replenishment optimization can outperform manual heuristics. By minimizing total inventory costs and eliminating the sources of labour intensive reconciliation, the research aims to shift the organization from reactive stock management toward a proactive, lean operational model (Pinto & Ferreira, 2021; Vicente, 2025).

### 1.4.Proposed Solution

This research proposes an integrated ML–MILP prescriptive framework for optimizing inventory replenishment in packaging operations. The solution adopts a Forecast then Optimize pipeline: the first stage employs a Machine Learning (ML) predictive model to generate demand forecasts for high variance, fragmented sales patterns. These forecasts serve as stochastic inputs to the second stage, a Mixed-Integer Linear Programming (MILP) optimization engine. The MILP model determines reorder quantities and timing while explicitly incorporating holding costs, fixed ordering fees and operational constraints such as storage capacity. By leveraging forecasted demand as input, the model generates cost optimal replenishment decisions that align inventory levels with anticipated demand patterns.

The proposed solution is strictly analytical, emphasizing the evaluation of the hybrid policy through controlled simulation experiments rather than commercial system deployment. Model performance will be benchmarked against existing manual, fixed-interval heuristics using metrics such as total operational cost, inventory stability, and stock count accuracy. By formalizing inventory decisions within a mathematically defined optimization framework, this research seeks to substitute labour intensive and reactive stock adjustments with a systematic, evidence driven decision model designed to minimize inefficiencies linked to periodic inventory reconciliation. This shift from manual control to algorithmic management enables inventory levels to align with real-time demand patterns instead of relying on subjective projections.

## 1.5.Objectives

### 1.5.1 General Objective

To develop and evaluate an integrated Machine Learning (ML) and Mixed-Integer Linear Programming (MILP) framework for optimizing inventory replenishment and minimizing operational inefficiencies in packaging material supply operations.

### 1.5.1. Specific Objectives

1. To examine manual inventory replenishment practices used by packaging material suppliers by identifying ordering rules, reconciliation frequency and operational constraints such as storage capacity
2. To develop a Machine Learning predictive model to forecast high-variance demand for fragmented packaging products using historical sales data.
3. To formulate a Mixed-Integer Linear Programming (MILP) model that integrates ML-generated demand forecasts to determine optimal replenishment quantities and timing while accounting for cost structures and operational constraints.
4. To benchmark the performance of the integrated ML-MILP framework against manual heuristic policies through simulation to evaluate cost-efficiency and inventory stability.

## 1.6.Research Questions

1. What inventory replenishment practices and reconciliation procedures characterize manual inventory management among packaging material suppliers?
2. How effectively can Machine Learning algorithms predict non-linear demand patterns for fragmented packaging products?
3. How can an MILP model be formulated to integrate ML-driven demand forecasts with cost and operational constraints to optimize inventory replenishment decisions?
4. To what extent the integrated ML-MILP framework improve total inventory cost, inventory level stability and stock-count accuracy relative to current manual practices?

## 1.7.Justification

The rationale for this research stems from the ongoing inefficiencies in inventory management among small and medium sized enterprises, especially those dependent on manual and heuristic decision processes. These approaches frequently lead to irregular replenishment, inaccurate inventory records and labour intensive reconciliation activities. By applying a hybrid framework that integrates Machine Learning (ML) with a mathematically structured optimization framework, the study seeks to bridge a practical operational gap while promoting evidence based practices in inventory control.

From an academic perspective the study contributes to the field of Data Science and operations research by demonstrating the practical integration of predictive and prescriptive analytics. Specifically, it shows how machine learning based demand forecasting can be effectively combined with Mixed-Integer Linear Programming (MILP) to support data driven inventory replenishment decisions. It further enriches the inventory optimization literature by situating MILP-based decision frameworks within SME supply contexts that remain insufficiently explored in empirical research.

From a practical standpoint, the proposed solution offers business owners and operations managers a systematic substitute for reactive inventory practices. The results are anticipated to enhance decision consistency, minimize operational inefficiencies and lessen dependence on labour intensive manual adjustments. In addition, the study reflects ISO 9001:2015 quality management standards by fostering data driven decision making and continuous improvement in processes.

## 1.8.Proposed System Methodologies

This research adopts an Iterative and Incremental Development approach, consistent with Agile principles, to guide the development of the proposed system. The choice of this methodology is justified by its iterative structure and its emphasis on progressively refining system components to align with organizational requirements. This ensures that the final hybrid framework effectively addresses demand variability and supports data driven inventory decision-making within the packaging sector.

The lifecycle begins with data understanding and preparation, where the Pandas library is employed to clean and standardize historical sales records. This foundational phase ensures that subsequent models are trained on high integrity datasets that accurately capture the fragmented demand patterns characteristic of the packaging sector.

The implementation stage follows a Forecast then Optimize pipeline. Predictive modelling is conducted using XGBoost (Extreme Gradient Boosting), selected for its ability to manage high variance, non-linear demand spikes through sequential boosting and L2 regularization, thereby mitigating the risk of overfitting in noisy SME datasets. The resulting forecasts are then ingested by a Mixed-Integer Linear Programming (MILP) model, formulated via the PuLP library in Python, to determine optimal replenishment schedules. This dual tool approach is justified because it enables the system to address complex operational constraints such as storage limitations and cost parameters that cannot be resolved by predictive models alone.

The research concludes with a simulation based evaluation designed to benchmark the hybrid ML–MILP framework against existing manual heuristics. Controlled discrete event simulations will be used to validate performance, employing metrics such as total operational cost reduction, inventory level stability and stock-count accuracy. This quantitative approach ensures the study remains an analytical evaluation of mathematical policies, providing empirical evidence for reducing the labour intensive reconciliation cycles that currently hinder organizational efficiency.

## 1.9.Scope

The scope of this research is restricted to inventory replenishment decision making for packaging materials within a single operational setting. It concentrates on the interaction between demand forecasting (ML) and order optimization (MILP), specifically identifying optimal order quantities and timing, without extending to broader supply chain activities such as supplier selection, transportation, warehouse optimization or distribution management.

Additionally, the study is limited to assessing a mathematical optimization model through simulation rather than implementing a real time system or enterprise wide solution. Constraints related to data availability, demand forecasting accuracy and parameter

assumptions are recognized. The results will be interpreted within the specific operational and methodological boundaries established for the study.

# CHAPTER 2: LITERATURE REVIEW

## 2.1. Introduction

This chapter examines the literature on inventory management and optimization, with a focus on data driven decision models used in inventory replenishment and supply chain planning. It outlines the fundamental concepts and methodologies that inform the project, including traditional inventory control practices, optimization approaches such as Mixed-Integer Linear Programming (MILP) and machine learning based demand forecasting techniques.

The purpose of this review is to establish the theoretical foundation for the research and to identify gaps in current knowledge that this study seeks to address. In particular, this chapter examines the evolution of predictive analytics (machine learning) and prescriptive analytics (mathematical optimization) as complementary approaches to enhancing inventory decision making. It assesses the ability of these approaches to enhance demand estimation, decision quality and operational performance when compared to heuristic or rule based approaches. By situating the study within this body of knowledge, the chapter underscores both the practical relevance and academic contribution of adopting hybrid ML informed MILP frameworks in modern inventory management.

## 2.2 Theoretical Review

### 2.2.1 Inventory Management Concepts

Inventory management involves the planning and control of raw materials, work in progress and finished goods to ensure product availability while minimizing overall operational costs. Core variables frequently addressed in optimization literature include demand, replenishment quantity, ordering frequency, holding cost, ordering cost and stock out penalties (Zhang & Huang, 2020).

Demand represents the quantity of items required by customers within a given period and may be deterministic or stochastic. In small to medium enterprises (SMEs), demand is typically uncertain and influenced by seasonal fluctuations and customer specific factors (Nguyen & Shiraki, 2021). Holding cost refers to expenses associated with storing inventory, including warehousing, handling and capital tie-up. Ordering cost encompasses fixed administrative and transportation expenses incurred whenever an order is placed, regardless of size. Stock-out costs capture the losses incurred when inventory fails to meet demand, including lost sales and diminished customer satisfaction (Zhang, X., & Huang, Y. 2020).

### 2.2.2 Inventory Control Policies

Inventory control policies establish rules for replenishment decisions. Traditional approaches include Economic Order Quantity (EOQ), periodic review and reorder point systems. While the Economic Order Quantity (EOQ) provides a foundational framework for cost minimization, its practical utility is constrained by assumptions of constant demand and the absence of storage capacity or integer constraints that do not hold under real operational conditions limiting its applicability in realistic, variable environments (Essien, E.E.& Otu, U.O.E. 2024). Heuristic and rule based policies remain common in SMEs due to their simplicity. These rely on judgment, fixed reorder intervals or visual stock checks. While straightforward to implement, such methods often fail to minimize costs under demand variability and operational constraints (Nguyen & Shiraki, 2021).

Optimization-based policies, particularly those employing Linear Programming (LP) and MILP, provide a prescriptive alternative. MILP models incorporate discrete decision variables, fixed ordering costs, storage capacity constraints, and multiple time periods, making them well suited to real world inventory systems (Pinto & Ferreira, 2021).

### 2.2.3 Optimization and MILP in Inventory Decision Making

Optimization techniques address decision problems by identifying the best possible outcome according to a defined objective function subject to constraints. In inventory management, the objective function typically seeks to minimize total costs including holding, ordering, and stock-out penalties while ensuring demand satisfaction and adherence to operational limits. Mixed-Integer Linear Programming (MILP) extends traditional linear programming by incorporating integer and binary variables, thereby enabling the modelling of discrete decisions such as whether to place an order and how to account for capacity restrictions. Research on MILP models for inventory and supply chain planning illustrates how discrete scheduling and ordering decisions can be encoded in mathematical constraints to minimize total costs in multi-period settings (Vicente, J. J. 2025).

MILP provides a prescriptive analytics framework that translates demand forecasts into actionable replenishment decisions by solving a structured optimization problem. Its key strengths include: Representation of both continuous decisions e.g., order quantities and discrete decisions e.g., order placement, Integration of operational constraints such as storage capacity and fixed ordering costs. Provision of optimal solutions under defined cost parameters.

### 2.2.4 Why Hybrid ML + MILP (Predictive–Prescriptive Analytics)

Predictive analytics which involves forecasting based on data and prescriptive analytics which involves optimization of decisions represent two complementary stages of data driven decision making. A predictive system focuses on anticipating future demand, while a prescriptive system determines the optimal response to that demand. Integrating these two approaches, commonly referred to as predictive–prescriptive analytics, is increasingly recognized in operations research as a robust framework for decision support (Bertsimas & Kallus, 2018).

Recent literature highlights the effectiveness of hybrid models in supply chain and inventory contexts. For instance, hybrid frameworks that combine demand forecasting with inventory decision optimization have been shown to improve performance relative to traditional policies, particularly under stochastic demand conditions. For instance, predict and optimize models consistently achieve lower total costs than periodic review approaches (Namazian, Z., Betts, J.M. & Stuckey, P.J 2025).Similarly, applied studies demonstrate that AI based forecasting followed by LP or MILP optimization improves service levels and reduces costs compared to traditional heuristic methods (Kumar & Goyal, 2025).

Integrating ML and MILP enables systems to employ machine learning to generate accurate demand forecast and apply MILP optimization to determine optimal order quantities and timing under cost and operational constraints. This hybrid architecture reflects current industry trends toward data driven, automated decision systems, positioning predictive–prescriptive analytics as an emerging technology in supply chain and inventory optimization.

## 2.3 Case Study Review

Empirical applications of inventory optimization models highlight their relevance in operational contexts, particularly SMEs and supply oriented businesses. Nguyen and Shiraki (2021) found that SMEs often rely on manual heuristics despite frequent stock-outs and excess inventory. Their study showed that optimization-based policies significantly reduced inventory costs compared to fixed-interval replenishment methods. Simulation-optimization approaches have also been explored. Yousefi and Lari (2022) integrated MILP decision rules with stochastic demand scenarios, demonstrating improved inventory stability and cost reduction relative to heuristics, though implementation complexity remained a limitation

Research focusing on African small enterprises further highlights limited adoption of mathematical decision support tools. Munyoka and Doyo (2022) examined African small enterprises and identified challenges including inconsistent record keeping, demand uncertainty and reactive inventory decision making, conditions that closely mirror packaging material suppliers. While some operational inefficiencies such as unit of measure management have been partially addressed through point of sale systems, the absence of demand forecasting and mathematically optimized replenishment policies remains a persistent challenge. These constraints often discourage the use of structured optimization despite their potential benefits.

Recent studies highlight the importance of integrating predictive and prescriptive analytics. Hybrid methodologies that combine machine learning driven demand forecasting with optimization techniques have demonstrated greater adaptability to dynamic demand fluctuations, while simultaneously ensuring cost-effectiveness and adherence to operational constraints (Kumar & Goyal, 2025). Findings suggest that although MILP models are effective in producing optimal solutions, the inclusion of enhanced demand forecasts significantly improves overall system efficiency, particularly in contexts where demand is influenced by nonlinear trends or external factors.

Most existing case studies predominantly examine manufacturing or large scale retail supply chains. In contrast, small and medium-sized enterprises (SMEs) engaged in packaging material distribution, particularly those managing mixed procurement and sales formats such as rolls versus individual pieces and relying on manual stock reconciliation, have received comparatively little empirical investigation. This scarcity of context specific evidence underscores a research gap that the present work aims to fill by assessing a hybrid inventory framework that integrates machine learning based forecasting with MILP optimization in the SME packaging sector.

## 2.4 Integration and Architecture

Inventory optimization models are typically deployed as decision support tools rather than full transactional systems. In practice, MILP based systems are implemented using modular architectures comprising data pre-processing, optimization and performance evaluation components (Pinto & Ferreira, 2021). Historical sales and inventory data are standardized into parameters, after which the optimization model computes replenishment decisions. These are then evaluated using cost and stability metrics.

For SMEs, lightweight implementations of MILP are preferred. Python libraries such as PuLP support the formulation and solution of mixed-integer optimization problems with

open-source solvers, enabling developers to create discrete decision models for inventory replenishment, capacity limits and ordering decisions without requiring large enterprise systems. This supports simulation-based analysis while remaining accessible to resource constrained businesses.

The proposed hybrid ML–MILP framework advances existing architectures by coupling a predictive component with a prescriptive optimization engine. Historical records of sales and inventory are employed to train and validate forecasting models such as machine learning or time series approaches yielding demand projections that feed into the MILP optimization process. The MILP module subsequently determines cost-minimizing inventory strategies while adhering to operational requirements, including storage limitations, fixed ordering costs, and unit-of-measure conversions.

This end to end decision-support pipeline: forecasting module → optimization module → performance evaluation, constitutes a cutting-edge prescriptive analytics paradigm for data driven inventory management (Bertsimas & Kallus, 2018). By uniting predictive forecasting with optimization, the framework harnesses advanced computational methods to improve decision accuracy, mitigate inefficiencies, and equip SMEs with practical, evidence-based inventory policies.

## 2.5 Summary

Existing research demonstrates that inventory management is strengthened through the integration of predictive and prescriptive techniques. Machine learning contributes to improved forecasting precision, while MILP delivers optimal decisions that account for operational constraints. The convergence of these methods produces a comprehensive predictive–prescriptive analytics framework, which is increasingly acknowledged as a leading approach for advancing data-driven inventory management systems.

## 2.6 Research Gaps

Although interest in inventory optimization and hybrid decision-making frameworks has grown, notable research gaps persist. Firstly, few empirical investigations have explored the deployment of ML enhanced MILP models within small to medium packaging material suppliers, particularly in the context of developing economies. Second, even where operational tools such as point of sale systems address transactional inefficiencies, the integration of predictive analytics and prescriptive optimization for replenishment decision making remains insufficiently examined in SME contexts Third, comparative analyses evaluating the performance of ML–MILP hybrids against conventional heuristic or standalone optimization techniques in SME inventory settings are limited.

To address these gaps, the present study proposes the design and evaluation of a predictive–prescriptive inventory optimization framework that integrates machine learning based forecasting with MILP optimization, specifically tailored to a packaging material supplier. Through simulation driven experimentation, the research aims to demonstrate the practical benefits of combining forecasting and optimization in enhancing inventory stability, lowering operational costs and strengthening decision-making within SME operations.

# CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

## 3.1. Introduction

This chapter details the system analysis and design of the proposed Machine Learning driven and Optimization based Inventory Management System. Its purpose is to provide a structured account of the conceptualization process undertaken prior to implementation. The discussion outlines the adopted development methodology, assesses the feasibility of the system and specifies both functional and non-functional requirements derived from stakeholder input.

In addition, the chapter presents the logical and physical design models that define the system's architecture, data flows, processing logic and user interaction mechanisms. These analysis and design activities ensure that the resulting system effectively addresses the identified inventory management challenges while remaining technically feasible and operationally suitable within the intended environment.

## 3.2. System Development Methodology

The development of the proposed system adopted Iterative and Incremental approach consistent with Agile principles. Agile emphasizes flexibility, continuous refinement and the progressive delivery of functional components. Rather than adopting a full Agile framework such as Scrum, which is typically suited to larger collaborative teams—this project employed an iterative model to accommodate the experimental requirements of machine learning and optimization development. Each subsystem, including data pre-processing, demand forecasting and optimization, was designed and tested independently before integration into the overall architecture. This strategy enabled ongoing model refinement and minimized risks by facilitating early identification and resolution of errors.

In contrast to the traditional Waterfall model, which enforces a rigid sequential process, the iterative methodology supports phased development. The forecasting module (XGBoost) and the optimization module (MILP) were separately implemented, validated and refined prior to their integration into a unified decision-support framework.

The methodology comprised the following phases:

- Requirements elicitation and analysis
- Prototype development of the forecasting model
- Validation and performance evaluation
- Development of the optimization model
- Integration of forecasting and optimization modules
- System testing and refinement

## 3.3 Feasibility Study

A comprehensive feasibility assessment was undertaken to evaluate the practicality of the proposed system across technical, economic, operational and scheduling dimensions.

### 3.3.1 Technical Feasibility

The system is technically viable as it is implemented in Python, an open source programming language with extensive support for data processing i.e., Pandas, NumPy, machine learning i.e., XGBoost and optimization modelling i.e., PuLP. These libraries are widely adopted and compatible with standard computing environments. The solution does not require specialized hardware and can run efficiently on a personal computer with moderate processing capabilities. Additionally, the structure of the available data supports pre-processing, forecasting, and optimization processes. Therefore, no specialized or unavailable technologies are required for implementation.

### 3.3.2 Economic Feasibility

From an economic standpoint, the system is cost effective since it leverages open source technologies, thereby avoiding licensing expenses. No additional hardware investment is necessary beyond existing infrastructure. At the organizational level, the system is expected to lower inventory holding costs, reduce stock-out risks, and enhance order planning efficiency. These improvements translate into measurable cost savings and operational benefits, supporting the justification for its development.

### 3.3.3 Operational Feasibility

Operational feasibility is ensured within the SME context, as the system is designed to be lightweight and user friendly. Minimal user interaction is required: operators simply upload sales and inventory datasets and initiate the forecasting and optimization processes. The system outputs structured reports aligned with current workflows, facilitating adoption without extensive training requirements.

### 3.3.4 Schedule Feasibility

The project timeline is consistent with the academic research schedule. The modular design of the system supports phased implementation and validation, ensuring that development milestones can be achieved within the allocated timeframe.

## 3.4 Requirements Elicitation

### 3.4.1 Data Collection Methods

The study employs a dual-method approach to data collection, integrating document analysis with semi-structured interviews to facilitate collection of both quantitative and qualitative data. This strategy ensures that the quantitative parameters required for the XGBoost and MILP models are grounded in the qualitative operational realities of the packaging supply environment.

#### 3.4.1.1 Document Analysis

Primary quantitative data is derived from a systematic analysis of historical inventory records, specifically stock movement ledgers and monthly reconciliation reports. These documents provided transaction-level data (IN and OUT quantities), inventory balances and historical demand patterns. Each document is rigorously audited for proper structure, consistency and data completeness to ensure the resulting dataset provides a statistically sound basis for predictive modelling and demand pattern analysis.

#### 3.4.1.2 Semi-structured Interviews

To complement the quantitative data, semi-structured interviews are conducted with key personnel responsible for inventory oversight and procurement decision making. These interviews are designed to capture nuanced qualitative insights into current inventory management practices, decision making processes, user expectations for an automated system as well as challenges and inefficiencies faced in current operations that is the prevailing inventory replenishment heuristics, the cognitive logic behind current decision making processes and the specific drivers of operational inefficiencies. Furthermore, this engagement facilitates the identification of user defined functional requirements and expectations for the proposed framework. The qualitative evidence gathered serves to contextualize the operational environment within which the system will be deployed, particularly in relation to real world constraints such as storage limitations and existing system capabilities.

Findings from the interviews revealed that unit of measure conversion, specifically the Bulk to Break transformation, is currently automated within the organization's point of sale (POS) system. This functionality is embedded at the transactional level and eliminates the need for manual conversion during day to day operations. However, the interviews also indicated that while such conversions are handled operationally, they are not explicitly integrated into higher level analytical processes such as demand forecasting and inventory optimization. As a result, the proposed framework focuses on leveraging already standardized transactional data as input for predictive and prescriptive modelling. A standardized interview guide, detailed in Appendix E, is utilized to ensure thematic consistency across all respondent sessions.

### 3.4.2 Sampling Technique and Sample Size

This study adopts a purposive sampling technique, specifically targeting individuals with direct oversight of inventory operations. The sample includes key stakeholders such as inventory clerks, store managers, and procurement personnel who possess the specialized knowledge required to describe existing inventory management practices and system usage within the organization. The respondents contributed valuable perspectives on the utilization of existing systems, particularly the role of the point of sale (POS) platform in automating selected inventory processes. These insights helped shape the boundaries and priorities of the proposed framework by clarifying how current technological capabilities influence operational efficiency. Given the exploratory nature of this research and the high level of technical expertise needed to understand the existing manual heuristics, a small, focused sample size is considered appropriate. This ensures that the qualitative insights gathered are high quality and directly relevant to the operational bottlenecks being studied.

### 3.4.3 Relevance of Data

The data collected through the aforementioned methods is strategically aligned with the core objectives of the framework. Specifically, the longitudinal stock movement records serve as the primary features for training the XGBoost demand forecasting model. Similarly, the documented inventory balances and warehouse dimensions provide the physical boundaries and cost parameters necessary to populate the MILP optimization constraints. Furthermore, the qualitative evidence gathered through interviews proved essential in clarifying existing decision making practices and system utilization. Notably, respondents indicated that unit of measure conversions are automated within the organization's point of sale (POS) platform at the transactional stage. This observation supports the study's reliance on standardized data

inputs, enabling the proposed framework to concentrate on advanced analytical functions such as demand forecasting and inventory optimization rather than routine operational data transformations.

### 3.4.4 Use of Data in Requirements Derivation

The gathered data serves as the foundation for deriving the system's functional and non-functional requirements. By identifying the specific points of failure in current manual practices, such as late stage reconciliation errors and overstocking, the study can define the precise inputs and outputs required for the Python based simulation. Furthermore, this data helps establish the performance benchmarks for the framework, ensuring that the final model is not only mathematically sound but also practically viable within a high variance packaging environment. This evidence driven approach ensures that the resulting requirements are grounded in actual operational needs rather than theoretical assumptions.

## 3.5 Data Analysis

Quantitative data extracted from the organization's stock movement records was analysed using Microsoft Excel to identify demand patterns and inform the design of the forecasting and optimization models.

### 3.5.1 Analysis Techniques

The OUT column from the stock movement records was aggregated by date and item to derive periodic demand values. Descriptive statistics including mean demand, standard deviation, and minimum and maximum values were computed to characterize demand variability across product lines. Trend analysis was subsequently performed to identify temporal patterns including seasonality and irregular fluctuations in demand behaviour.

### 3.5.2 Data Visualization

The following visualization approaches were applied to represent analytical findings: Line graphs were used to illustrate demand trends over the observation period, enabling identification of temporal patterns and irregularities. Bar charts were used to compare stock inflow (IN) and outflow (OUT) volumes across periods, highlighting imbalances between replenishment and consumption.

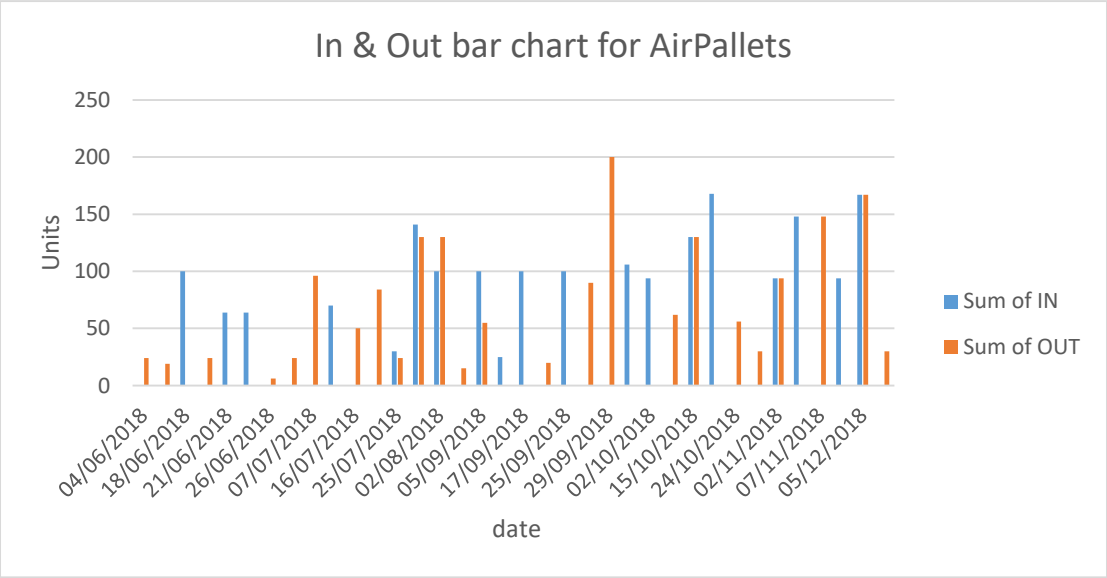


*Figure 1: Bar chart*

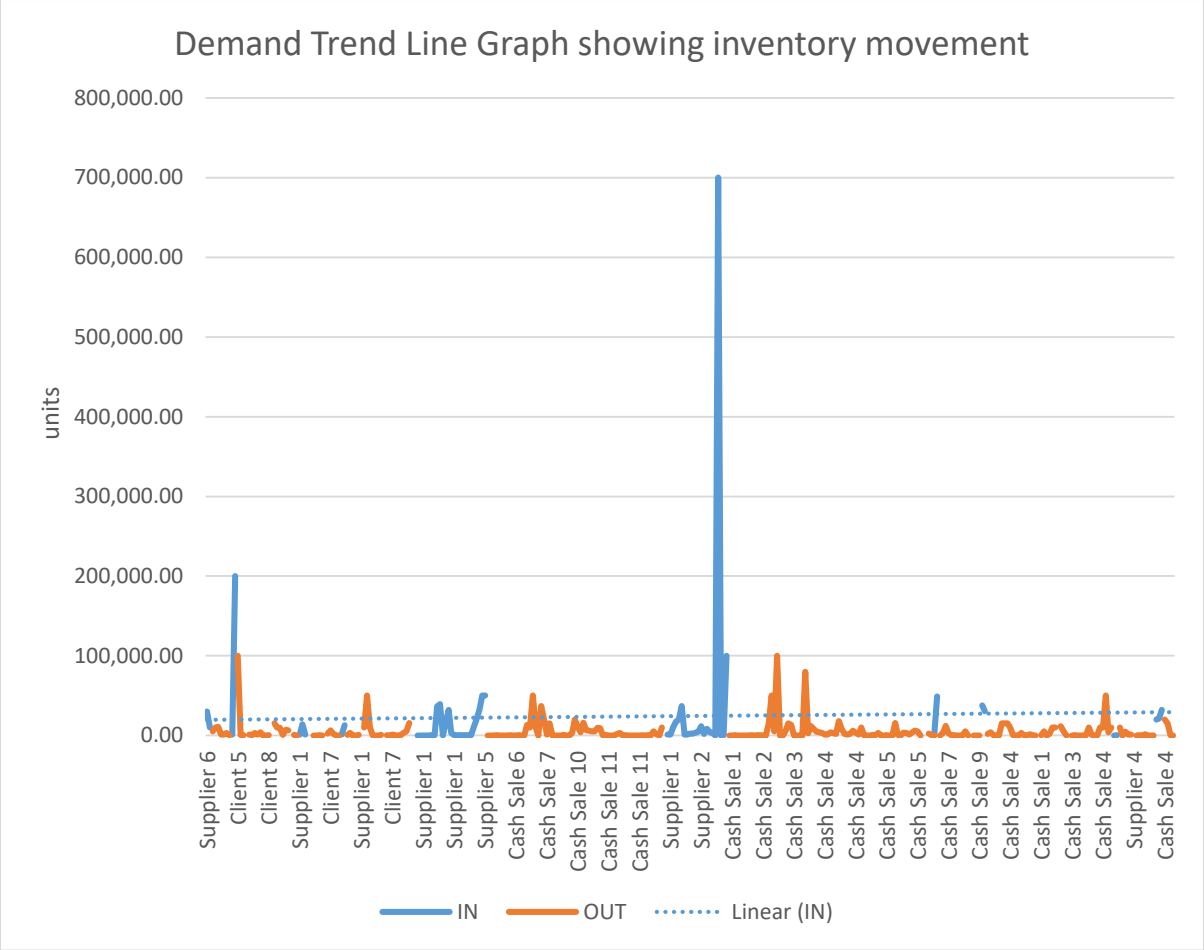


*Figure 2: Demand Line Graph*

3.5.3 Purpose of Analysis

The analysis serves two primary functions. First, it provides empirical evidence of demand variability and trend behaviour, which directly informs feature engineering for the machine learning forecasting model. Second, it establishes baseline inventory dynamics that are incorporated as parameters and constraints within the MILP optimization model.

## 3.6 System Specification

3.6.1 Functional Requirements

The following functional requirements were derived from document analysis and stakeholder engagement. Each requirement defines a specific system behaviour expected under normal operating conditions.

FR1 The system shall accept inventory transaction data in Microsoft Excel format (.xlsx).

FR2 The system shall validate uploaded files for the presence of required fields: date, item name, quantity in, and quantity out.

FR3 The system shall pre-process raw transaction data by handling missing values, standardizing date formats, and aggregating demand by period.

FR4 The system shall extract demand values from OUT transaction records and generate a structured time-series dataset.

FR5 The system shall train a machine learning model on historical demand data to generate future demand forecasts.

FR6 The system shall compute current inventory levels from transaction history.

FR7 The system shall formulate and solve a Mixed-Integer Linear Programming model to determine optimal reorder quantities and timing.

FR8 The system shall export optimization results, including forecasted demand and recommended order quantities, to an Excel file.

FR9 The system shall support independent analysis of individual inventory items.

### 3.6.2 Non-Functional Requirements

NFR1 The system shall produce results within two minutes of execution on standard computing hardware.

NFR2 The system shall provide a graphical user interface operable by non-technical users without programming knowledge.

NFR3 The system shall validate input data and return informative error messages for incorrectly formatted or incomplete files.

NFR4 The system shall maintain data accuracy throughout all processing stages, ensuring that output values are traceable to source records.

NFR5 The system shall operate as a standalone desktop application without requiring an active internet connection.

NFR6 The system shall be designed to accommodate additional inventory items without requiring structural modifications to the codebase.

## 3.7 Requirements Analysis and Modelling

### 3.7.1 Requirements Analysis

Analysis of collected data and stakeholder insights reveals several key characteristics of the current inventory management process. Replenishment decisions are made manually without reference to historical demand trends, resulting in reactive rather than proactive ordering. Although the point of sale (POS) system captures and standardizes transactional data, this information is not systematically utilized for forecasting or optimization purposes. Demand variability is not formally quantified, and decision making remains dependent on subjective judgment and periodic stock reviews. These findings confirm the need for a system that automates demand estimation and provides mathematically grounded replenishment recommendations.

### 3.7.2 Requirement Dependencies and Conflicts

The following dependencies were identified among system requirements:

FR5 (machine learning forecasting) is dependent on FR3 and FR4, as demand forecasting cannot be performed without pre-processed, structured demand data. FR7 (MILP

optimization) is dependent on FR5 and FR6, as the optimization model requires both a demand forecast and a current inventory level as input parameters. FR8 (output generation) is dependent on the successful execution of FR7.

No direct conflicts were identified among the specified requirements. However, a potential tension exists between NFR1 (execution speed) and FR5 (model training), as machine learning model training may introduce latency for large datasets. This is addressed by caching the trained model after initial execution and retraining only when new data is provided.

### 3.7.3 System Modelling

The following modelling tools are used to represent system requirements and behaviour. The actual diagrams are presented below each description.

a) Use Case Diagram

The use case diagram identifies the primary actor as the Inventory Manager, who interacts with the system through four core use cases: uploading inventory data, initiating the forecasting process, reviewing optimization results, and exporting the recommended order plan. The diagram illustrates that all use cases are accessible through the graphical interface and that forecasting and optimization are dependent on prior data upload.

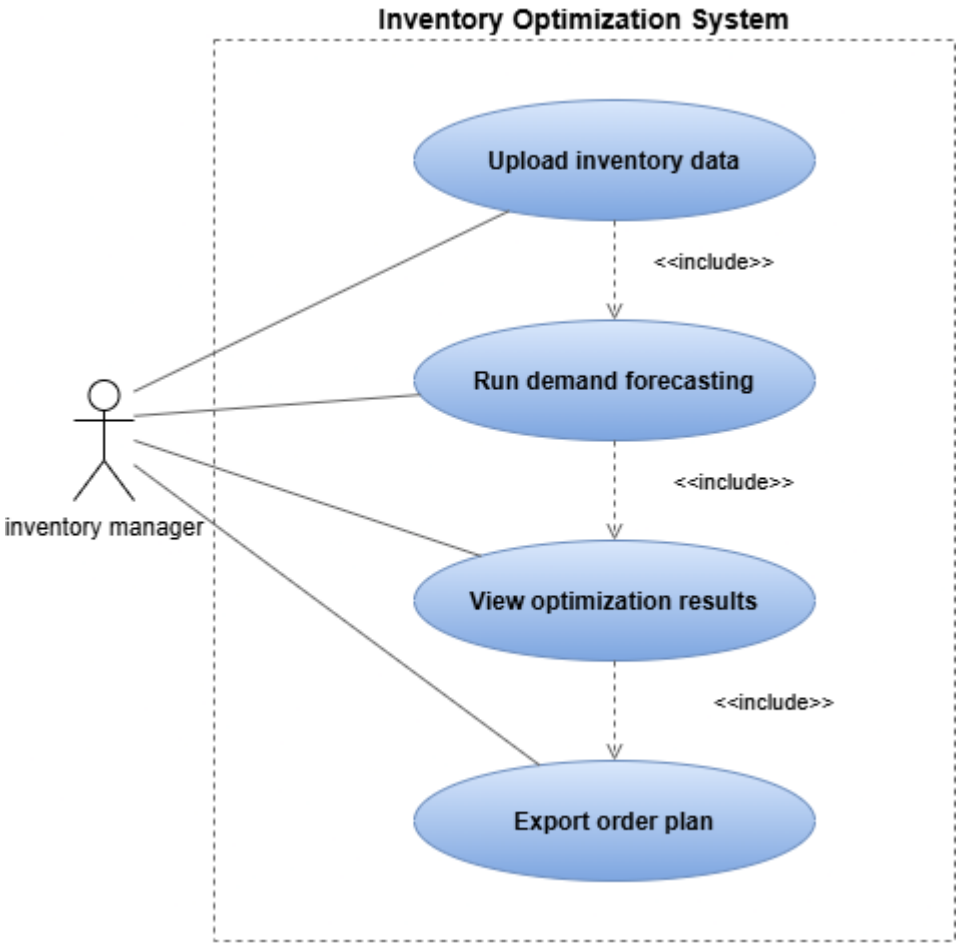


*Figure 3: Use Case Diagram*

b) Data Flow Diagram (DFD)

The Level 0 DFD represents the system as a single process that receives stock movement data from the inventory manager and returns an optimized order recommendation. The Level 1 DFD decomposes this into four sub-processes: data pre-processing, demand forecasting, inventory optimization, and result output. Data stores include the inventory database and the output report file.

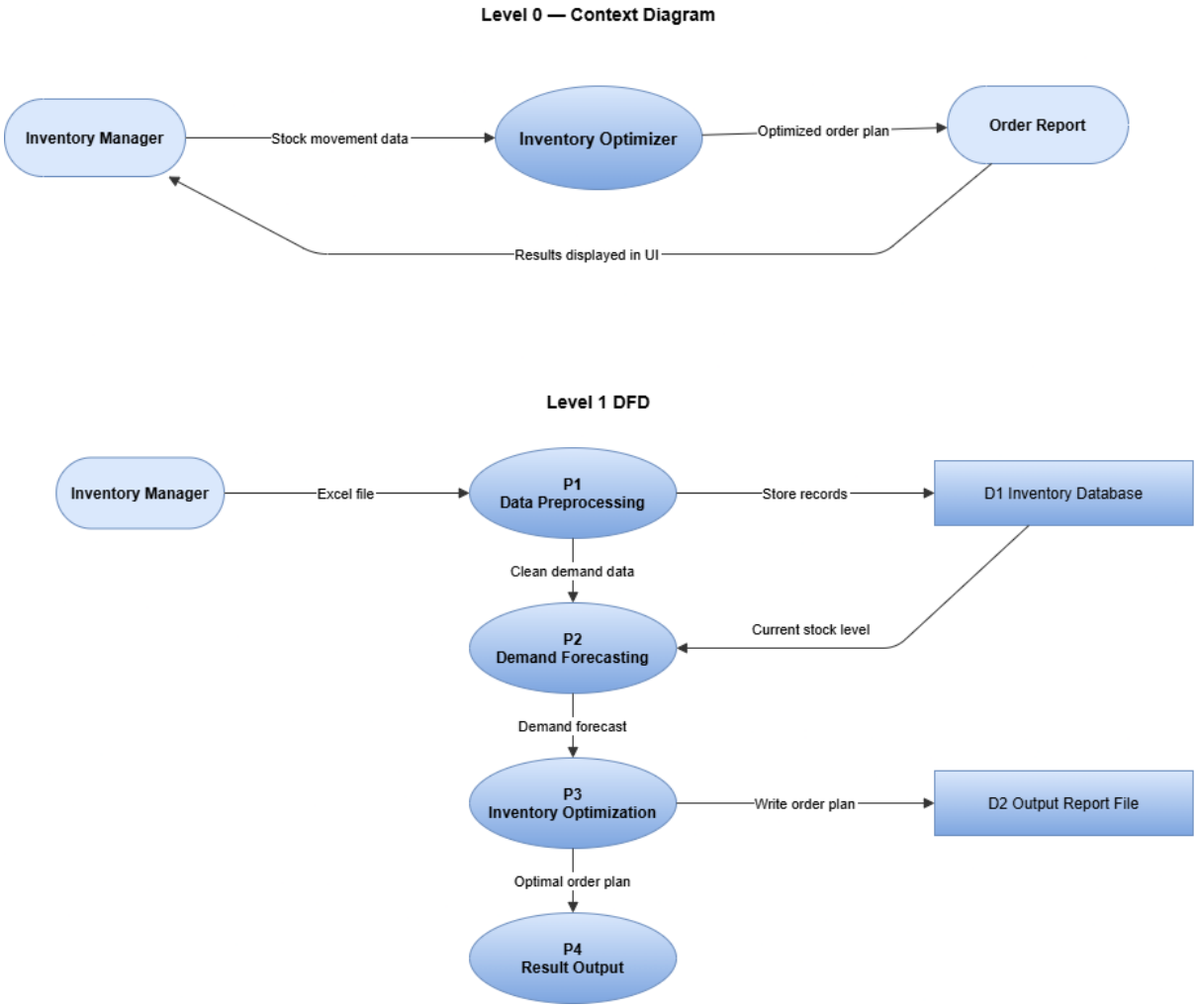


*Figure 4: Level 0 and Level 1DFD*

c) Activity Diagram

The activity diagram models the sequential execution of system operations from file upload through to result generation. Decision nodes are included to handle invalid input files and cases where the optimization model returns no feasible solution.

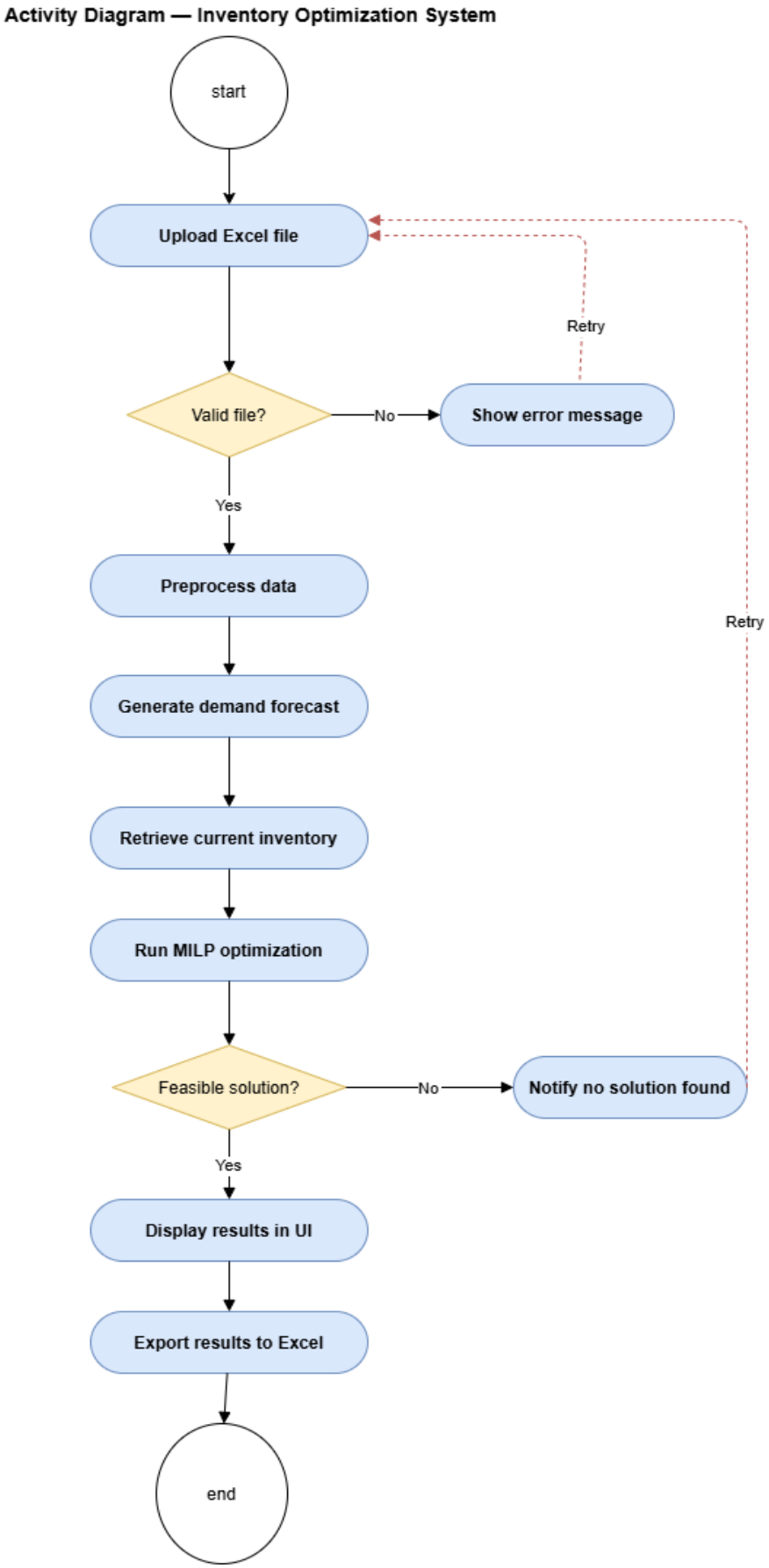


*Figure 5: Activity Diagram*

## 3.8 Logical Design

### 3.8.1 System Architecture

The system adopts a layered architecture comprising five distinct layers, each with a defined responsibility. This architectural pattern was selected because it promotes modularity, simplifies testing, and allows individual layers to be modified without affecting the broader system.

User Interface Layer: Provides the graphical interface through which the inventory manager uploads data and views results. Implemented using the Python Tkinter library.

Application Logic Layer: Coordinates the execution sequence of the system, passing data between pre-processing, forecasting, and optimization components.

Machine Learning Module: Encapsulates the XGBoost forecasting model, including feature engineering, model training, and prediction generation.

Optimization Module: Contains the MILP model formulated using the PuLP library, responsible for computing optimal reorder decisions.

Data Layer: Manages storage and retrieval of inventory data using an SQLite database and Excel file I/O operations.

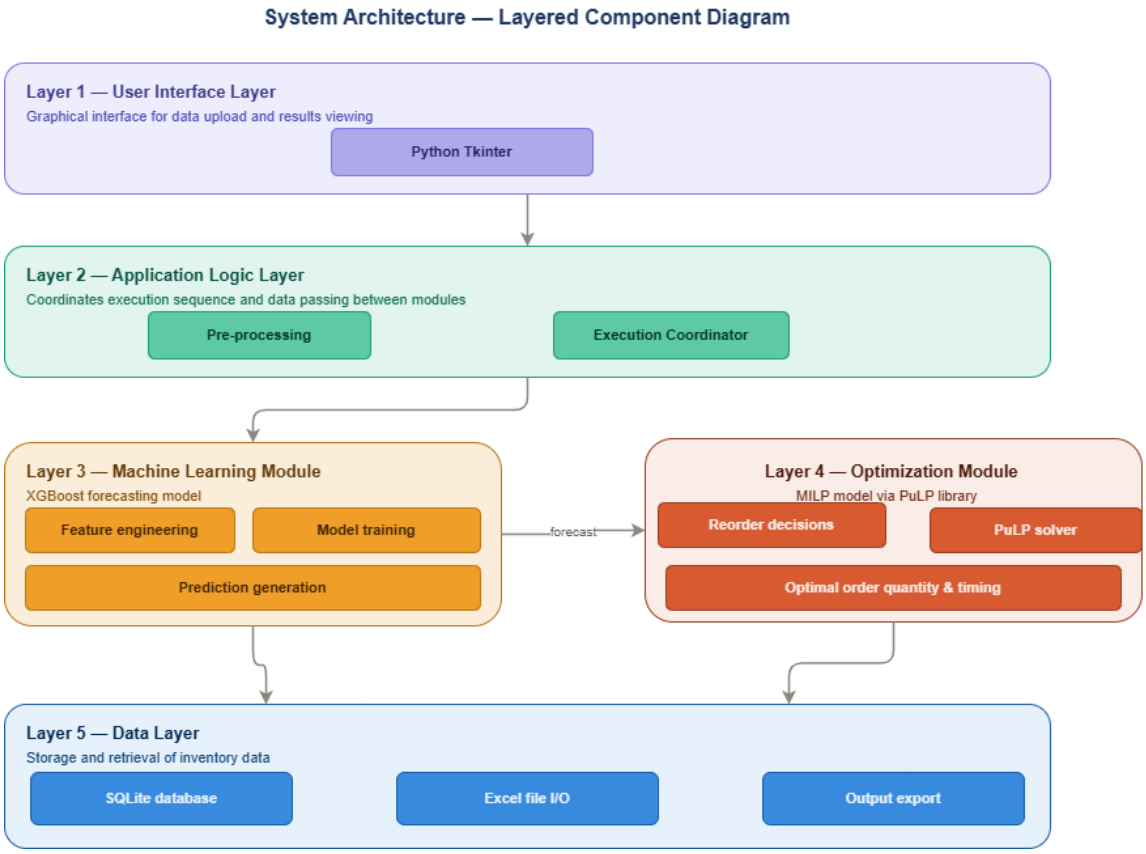


*Figure 6: System Architecture diagram*

3.8.2 Control Flow and Process Design

The system executes through the following sequential process: The inventory manager launches the application and selects an Excel data file through the graphical interface. The data pre-processing module loads the file, validates its structure, handles missing values, and generates time-series demand features. The machine learning module trains an XGBoost regression model on the processed demand history and generates a demand forecast for the next planning period. The optimization module receives the demand forecast and current inventory level as inputs and solves the MILP model to determine the optimal order quantity and timing. Results are written to an Excel output file and displayed within the user interface.

Pseudocode for Main Execution Pipeline:

```
BEGIN run_system(file_path)
    df ← preprocess(file_path)
    IF df is invalid THEN
        RAISE InputError
    END IF
    model ← train_model(df)
    demand ← predict(model, df.latest_features)
    inventory ← get_current_inventory()
    order ← optimize(demand, inventory)
    export_results(demand, order)
    RETURN demand, order
END
```

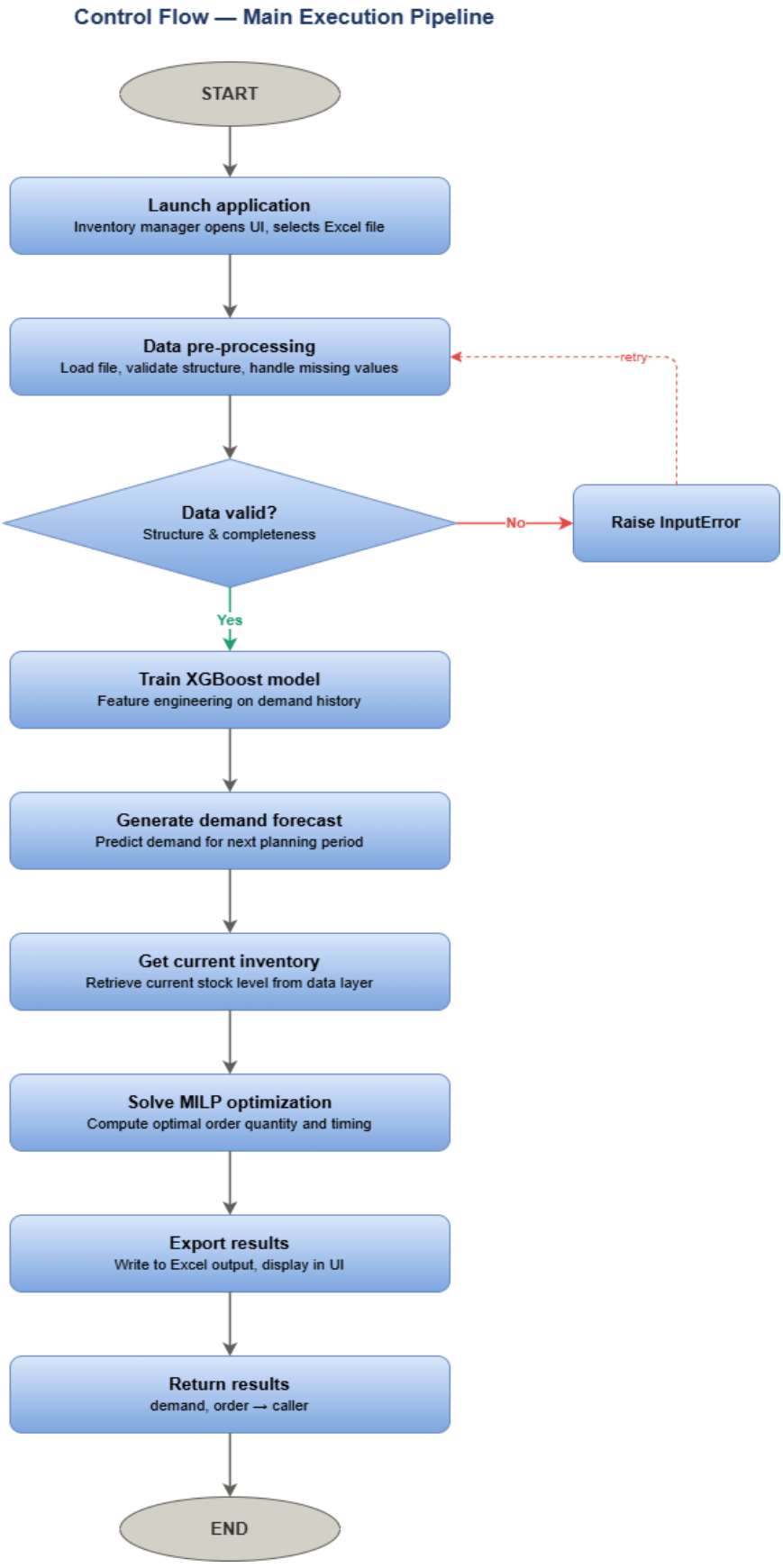


*Figure 7: Flowchart*

### 3.8.3 Design for Non-Functional Requirements

Security and Data Integrity: The system operates locally without transmitting data to external servers, ensuring that sensitive inventory records remain within the organization's computing environment. Input validation is performed prior to processing to prevent erroneous data from propagating through the pipeline.

Error and Exception Handling: The system implements structured exception handling at each processing stage. File format errors, missing columns, and solver failures are caught and communicated to the user through descriptive interface messages rather than raw error outputs.

Efficiency: To reduce execution time, the trained XGBoost model is serialized and cached after initial training. Subsequent executions load the cached model unless new data requires retraining, significantly reducing processing time for repeat runs.

## 3.9 Physical Design

### 3.9.1 Database Design

SQLite was selected as the database management system due to its lightweight nature, zero-configuration deployment, and seamless integration with Python. It is appropriate for a single-user desktop application that does not require concurrent access or network connectivity.

The database schema is defined as follows:

Run Summary Table

| Field | Data Type | Constraints |
|---|---|---|
| id | INTEGER | PRIMARY KEY, AUTOINCREMENT |
| run_date | TEXT | NOT NULL |
| total_analyzed | INTEGER | NOT NULL |
| items_requiring_reorder | INTEGER | NOT NULL |
| items_sufficient | INTEGER | NOT NULL |

*Table 1: Run Summary Table*

Reorder Report Table

| Field | Data Type | Constraints |
|---|---|---|
| id | INTEGER | PRIMARY KEY, AUTOINCREMENT |
| run_id | INTEGER | NOT NULL, FOREIGN KEY → RunSummary.id |
| item_code | TEXT | NOT NULL |

| | | |
|---|---|---|
| item_description | TEXT | NOT NULL |
| unit_of_measure | TEXT | NOT NULL |
| forecasted_demand | REAL | NOT NULL |
| current_stock | REAL | NOT NULL |
| recommended_order | REAL | NOT NULL |
| urgency | TEXT | NOT NULL |
| reorder_by | TEXT | NOT NULL |

*Table 2: Reorder Report Table*

The run_id foreign key links every reorder recommendation to the specific run that generated it, enabling retrieval of complete historical reports by date.

### 3.9.2 User Interface Design

The user interface is designed to minimize interaction complexity for non-technical users. It consists of two primary screens:

Screen 1 — Data Upload: Presents a file selection control allowing the user to browse and select an Excel file. A validation indicator confirms whether the selected file meets the required format before processing is initiated.

Screen 2 — Results Display: Displays the forecasted demand value, recommended order quantity, and a status confirmation upon completion. An export button allows the user to save results to an Excel file.

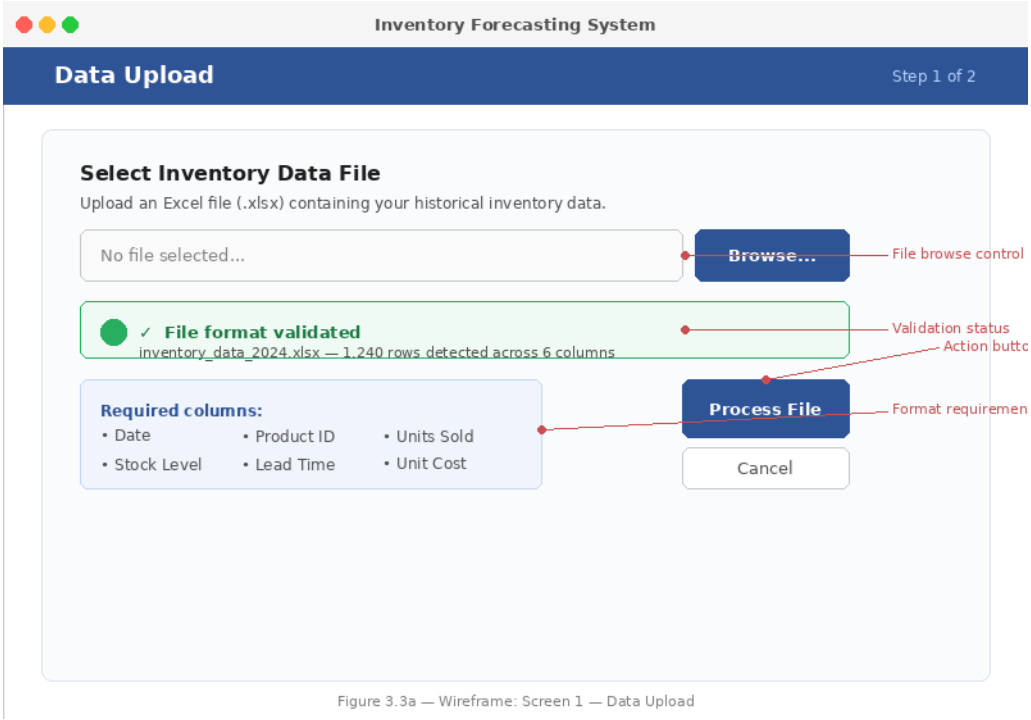


*Figure 8: Data Upload*

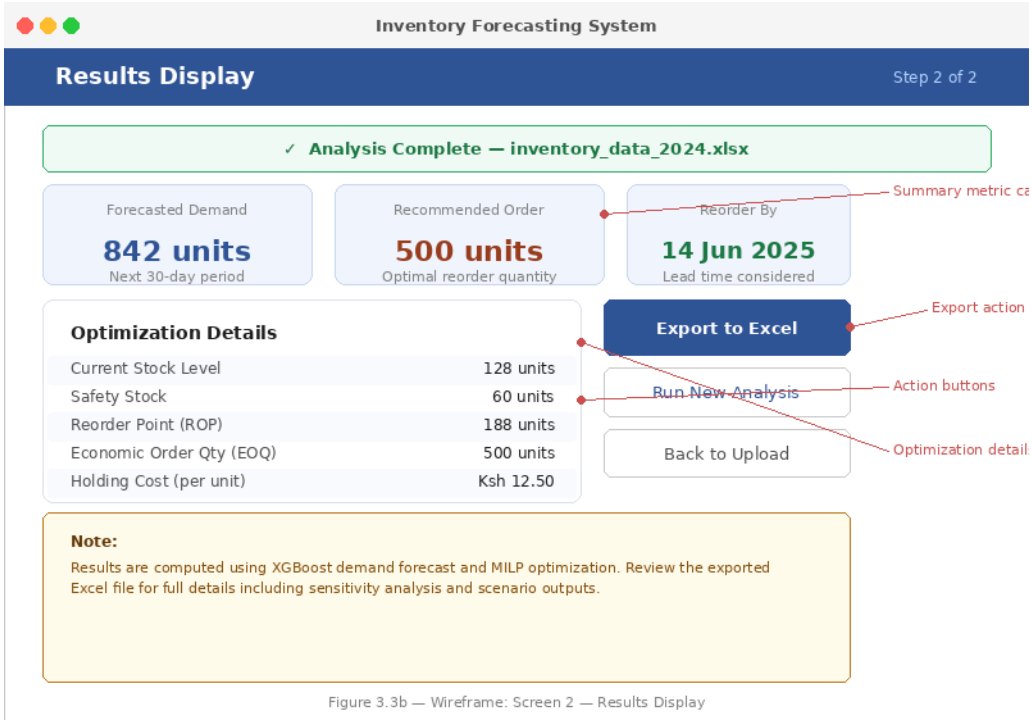


*Figure 9: Result display*

# Chapter 4: SYSTEM IMPLEMENTATION AND TESTING, CONCLUSIONS AND RECOMMENDATIONS

## 4.1 Introduction

This chapter provides a comprehensive account of the implementation, testing and assessment of the inventory optimization system. It outlines the development environment and tools utilized, highlights essential code segments that enable system functionality, and explains the testing methodologies adopted to ensure reliability and accuracy. The chapter concludes with a user manual, a synthesis of the research findings and recommendations to guide future advancements.

## 4.2 Environment and Tools

The system was developed and tested on a Windows 10 operating system using the following tools and technologies:

### Backend

Python 3.13 — primary programming language for data processing, machine learning, and optimization

pandas — data loading, cleaning, and aggregation of Excel-based inventory records

NumPy — numerical computation supporting safety stock and reorder point calculations

XGBoost — gradient boosting library for demand forecasting

scikit-learn — model evaluation metrics including MAE and RMSE

PuLP — Mixed-Integer Linear Programming library for inventory optimization

openpyxl — Excel report generation

### Frontend

Tkinter — Python standard library used to build the graphical user interface

threading — Python standard library used to run the optimization pipeline in a background thread, keeping the UI responsive during processing

### Data Layer

SQLite — lightweight relational database for structured data storage

Microsoft Excel (.xlsx) — primary input and output format for inventory data and recommendations

### Development Tools

Visual Studio Code — code editor

Git — version control

PowerShell — terminal for script execution and debugging

## 4.3 System Code Generation

The system is composed of seven modules, each responsible for a distinct stage of the pipeline. The following sections describe the key implementation decisions and present representative code snippets from each module.

### 4.3.1 Data Pre-processing (preprocessing.py)

The pre-processing module loads all monthly inventory sheets from the Excel workbook, detects and handles structural inconsistencies, cleans numeric columns, removes blank rows, and produces a unified stacked DataFrame. Monthly sheets are detected automatically by matching sheet names against known month names.

```
monthly_sheets = [
    s for s in all_sheets
    if any(m in s.strip() for m in MONTH_NAMES)
]


for sheet in monthly_sheets:
    df = pd.read_excel(excel_file, sheet_name=sheet)
    df.columns = df.columns.str.strip()
    df = df.loc[:, ~df.columns.str.contains('^Unnamed')]
    df['Month'] = sheet.strip()
    all_data.append(df)
```

Monthly aggregates are computed by grouping transactions by item and month, summing IN and OUT values to produce one record per item per period. This aggregated dataset forms the input to the forecasting module.

### 4.3.2 Safety Stock and Reorder Points (safety_stock.py)

Safety stock is calculated using the standard statistical formula SS = Z * sigma * sqrt(LT), where Z represents the service level factor set at 1.645 for a 95% service level, sigma is the standard deviation of historical monthly demand, and LT is the lead time expressed as a fraction of the planning period. A uniform lead time of four days was applied, consistent with observed supplier delivery patterns.

```
def calculate_safety_stock(demand_series):
    sigma = demand_series.std()
    if pd.isna(sigma) or sigma == 0:
        sigma = demand_series.mean() * 0.1
    return round(Z_SCORE * sigma * np.sqrt(LEAD_TIME_MONTHS), 2)


reorder_point = (avg_daily_demand * LEAD_TIME_DAYS) + safety_stock
```

The reorder point determines the threshold below which the MILP model flags an item for replenishment. Safety stock and reorder point values are computed internally and are not exposed in the output report.

4.3.3 Demand Forecasting (forecast.py)

The forecasting module trains an XGBoost regression model for each inventory item using time-series features derived from historical monthly demand. Features include three lag variables (lag1, lag2, lag3), a three-period rolling mean, and a three-period rolling standard deviation. Items with fewer than four months of data are assigned their historical mean as a fallback forecast.

```
df['lag1'] = df['demand'].shift(1)
df['lag2'] = df['demand'].shift(2)
df['lag3'] = df['demand'].shift(3)
df['rolling_mean_3'] = df['demand'].shift(1).rolling(3).mean()
df['rolling_std_3']  = df['demand'].shift(1).rolling(3).std().fillna(0)

model = XGBRegressor(n_estimators=100, learning_rate=0.1, max_depth=3)
model.fit(X_train, y_train)
forecast = max(model.predict(next_features)[0], 0)
```

Of 221 total items, 207 were forecasted using the trained XGBoost model and 14 were assigned mean demand values due to insufficient history. Forecast accuracy was evaluated using Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE).

4.3.4 MILP Inventory Optimization (milp_model.py)

The optimization module formulates a single-period Mixed-Integer Linear Programming model for each item. The objective is to minimize total inventory-related cost comprising holding cost, fixed ordering cost, and stock-out penalty. A binary decision variable Y determines whether an order is placed, enforced through the Big-M constraint Q <= M * Y.

```
Q = LpVariable('order_quantity', lowBound=0, cat='Continuous')
Y = LpVariable('order_decision', cat='Binary')
S = LpVariable('shortfall',      lowBound=0, cat='Continuous')

model += holding_cost + ORDERING_COST * Y + STOCKOUT_PENALTY * S

model += current_stock + Q - forecasted_demand + S >= 0  # balance
model += Q <= M * Y                                       # activation
model += current_stock + Q >= safety_stock                # buffer
```

Items whose current stock exceeds the reorder point are excluded from the solver to reduce computation. Of 221 items analyzed, 93 were identified as requiring replenishment in the next planning period.

4.3.5 Report Generation (report_generator.py)

The report module generates a formatted Excel workbook using openpyxl. The output file begins directly with a column header row and presents only items flagged for reorder, sorted by urgency. Items are classified as URGENT (zero or negative stock), LOW (stock below

30% of forecasted demand), or MONITOR (stock below reorder point). Auto-filter is applied to all columns to support managerial sorting and filtering.

#### 4.3.6 User Interface (ui.py)

The graphical interface is implemented using the Tkinter library. The UI presents a two-step workflow: file selection via a browse dialog followed by optimization execution via a single button. The pipeline runs in a background thread to prevent the interface from freezing during processing. Upon completion, a summary of results is displayed and the manager may open the generated report directly from the application.

#### 4.3.7 Database Layer (database.py)

The database module implements a lightweight SQLite persistence layer that records the outputs of each optimization run. On every pipeline execution, a summary record is inserted into the RunSummary table capturing the run date and item counts, and one record per flagged item is inserted into the ReorderReport table. This enables the organization to maintain a historical log of reorder recommendations across planning periods without requiring access to the original source files. The database is initialized automatically on first run and requires no configuration from the user.

## 4.4 Testing

A structured testing suite was developed and executed to validate system functionality, reliability, and usability. The following testing strategies were applied.

#### 4.4.1 Unit Testing

Unit tests were conducted on individual functions within each module to verify correct behaviour in isolation. Each function was tested with valid inputs, edge cases (zero demand, missing data), and invalid inputs. Test outcomes were recorded and compared against expected results.

#### 4.4.2 Integration Testing

Integration testing verified that data flows correctly between modules. The full pipeline was executed end-to-end to confirm that outputs from preprocessing served correctly as inputs to forecasting, and that forecast outputs fed into the MILP model without data loss or type errors.

#### 4.4.3 Black Box Testing

Black box testing was conducted by supplying the system with the actual historical inventory dataset and evaluating output correctness without reference to internal code logic. The reorder recommendations generated were reviewed against manually computed expectations to confirm alignment.

#### 4.4.4 Usability Testing

The system was tested with a non-technical user who simulated the role of an inventory manager. The user was asked to select the input file and execute the optimization without any coaching. The user completed the task successfully within two minutes, confirming that the interface is accessible without technical knowledge.

#### 4.4.5 Performance Testing

Performance testing evaluated the system's response time under realistic data load. With 215 unique items across 13 months (2,684 rows), the full pipeline including data loading, model

training, optimization, and report generation completed within approximately 45 seconds on standard computing hardware. This was considered acceptable for a periodic planning tool executed monthly.

### 4.4.6 Smoke Testing

A smoke test was performed at the start of each development iteration to verify that the core pipeline executed without critical failure after each code change. This involved running the main pipeline with a reduced dataset of 10 items to confirm that all modules executed and produced output before full-scale testing was conducted.

### 4.4.7 Test Cases

A total of 22 test cases were generated covering all seven system modules. Each test case specifies a unique identifier, the module under test, a description of the condition tested, the input supplied, the expected output, the actual output observed, and a pass or fail status.

| **TC ID** | **Module** | **Test Description** | **Input** | **Expected Output** | **Actual Output** | **Status** |
|---|---|---|---|---|---|---|
| TC-01 | Pre-processing | Load valid Excel file with all monthly sheets | Stock Card - Main.xlsx | 15 sheets detected and loaded | 15 sheets detected and loaded | **PASS** |
| TC-02 | Pre-processing | Handle sheet with missing Item Code column | Sheet missing Item Code | Sheet skipped with warning message | Sheet skipped with warning message | **PASS** |
| TC-03 | Pre-processing | Clean blank and separator rows | Rows with empty Item Description | Blank rows removed from dataset | Blank rows removed from dataset | **PASS** |
| TC-04 | Pre-processing | Convert numeric columns from text | OUT column containing mixed types | All values converted to float64 | All values converted to float64 | **PASS** |
| TC-05 | Pre-processing | Aggregate monthly totals per item | 13 months of multi-item data | One row per item per month with summed OUT | One row per item per month with summed OUT | **PASS** |
| TC-06 | Safety Stock | Calculate safety stock for high-demand item | Angle Board 2.3M demand series | SS = 3,914.69 units | SS = 3,914.69 units | **PASS** |

| | | | | | | |
|---|---|---|---|---|---|---|
| TC-07 | Safety Stock | Handle zero-demand item | Item with all-zero OUT values | Safety stock = 0, no reorder flagged | Safety stock = 0, no reorder flagged | **PASS** |
| TC-08 | Safety Stock | Calculate reorder point correctly | Avg daily demand = 262.59, SS = 3,914.69 | ROP = 4,965.05 units | ROP = 4,965.05 units | **PASS** |
| TC-09 | Safety Stock | Get current stock from last month | November 18 closing balance per item | Correct closing balance retrieved per item | Correct closing balance retrieved per item | **PASS** |
| TC-10 | Forecasting | XGBoost forecast for item with sufficient history | 13 months of Bubble Wrap demand | Positive forecast returned with MAE and RMSE | Forecast: 17.98 rolls, MAE: 0.03 | **PASS** |
| TC-11 | Forecasting | Fallback to mean for item with insufficient history | Item with only 3 months data | Mean demand used as forecast with status label | Mean demand returned with fallback status | **PASS** |
| TC-12 | Forecasting | Forecast is never negative | Item with declining demand trend | Forecasted demand >= 0 | Forecasted demand = 0.01 (non-negative) | **PASS** |
| TC-13 | Forecasting | Feature engineering creates lag columns | Monthly demand series | lag1, lag2, lag3, rolling_mean_3, rolling_std_3 created | All 5 feature columns created correctly | **PASS** |
| TC-14 | MILP | No order recommended when stock sufficient | Current stock > reorder point | Recommended order = 0, status: Sufficient stock | Recommended order = 0 | **PASS** |
| TC-15 | MILP | Order recommended when stock below reorder point | Current stock < reorder point | Positive order quantity returned | Order quantity returned by solver | **PASS** |

| | | | | | | |
|---|---|---|---|---|---|---|
| TC-16 | MILP | Handle NaN values without crashing | Item with NaN in forecasted demand | Item skipped gracefully, no crash | Item skipped with Skipped status | **PASS** |
| TC-17 | MILP | Only items needing reorder appear in results | 221 total items optimized | Only items with Recommended Order > 0 in results_df | 93 items returned in results_df | **PASS** |
| TC-18 | UI | File browse dialog opens on button click | User clicks Browse button | File dialog opens, file path populates label | File dialog opened and path shown | **PASS** |
| TC-19 | UI | Run button disabled until file selected | App launched with no file | Run button is greyed out and unclickable | Run button disabled as expected | **PASS** |
| TC-20 | Report | Excel report generated with correct structure | 93 items requiring reorder | Excel file saved with headers, data rows, and filters | Report saved to output folder successfully | **PASS** |
| TC-21 | Report | Urgency correctly assigned per item | Item with current stock = 0 | Urgency = URGENT displayed in red | URGENT shown in dark red for zero-stock items | **PASS** |
| TC-22 | UI | Open Report button opens Excel file | Valid report in output folder | Excel file opens in default application | File opened via os.startfile on Windows | **PASS** |

*Table 3: Test Cases*

## 4.5 User Guide

### System Requirements

- Windows 10 or 11 operating system
- Microsoft Excel or compatible spreadsheet application to view the generated report

### Installation

Step 1: Copy the file Inventory Optimizer.exe into any folder on the computer.

Step 2: Copy the inventory Excel file Stock Card - Main.xlsx into the same folder.

No additional software, libraries, or configuration is required.

### Running the System

Step 1: Double-click Inventory Optimizer.exe to launch the application. The window will open automatically centred on the screen.

Step 2: If Windows displays a protected your PC warning, click More info then Run anyway. This message appears because the application is not distributed through a commercial publisher and does not indicate a security risk.

Step 3: Click Browse and navigate to the inventory Excel file.

Step 4: Click Run Optimization. The status bar will display progress messages as each stage completes. Do not close the application while it is running.

Step 5: When the status displays Optimization complete, the results summary will appear showing the total number of products analyzed and the number requiring reorder.

Step 6: Click Open Report to open the generated Excel report directly from the application.

**Understanding the Report**

The generated report presents all inventory items requiring replenishment. Each row contains the item code, item description, unit of measure, forecasted demand for the next month, current stock level, recommended order quantity, urgency classification, and reorder date. Items are sorted by urgency, with URGENT items (zero or negative stock) appearing first. The Excel auto-filter on the header row allows the manager to sort or filter by any column.

**Urgency Classifications**

- URGENT — current stock is zero or below zero. Order immediately.
- LOW — current stock is below 30% of forecasted demand. Order soon.
- MONITOR — current stock is below the reorder point. Plan ahead.

**Output Files**

The application generates two outputs automatically in the same folder as the executable:

- output/Inventory_Reorder_Report.xlsx — the reorder recommendations report
- inventory_optimizer.db — a database file storing a history of all previous runs. This file should not be deleted as it preserves the historical record of recommendations.

**Updating Data**

When new monthly inventory data becomes available, open the Excel file and add a new sheet for that month following the same column structure as existing sheets:
Item Code | Item Description | Unit of Measure | Opening Balance | In | Out | Closing Balance
Name the sheet consistently with existing months and the last two digits of the year, for example December 19. Save and close the file. The next time the manager runs the application and selects the file, the system will automatically detect the new sheet, include it in model training and generate updated recommendations reflecting the latest data. The system does not need to be reconfigured or reinstalled when new data is added.

## 4.6 Conclusions

This research sought to address persistent operational inefficiencies in inventory management within a packaging material supply business, specifically the absence of a structured, data driven replenishment decision framework and the resulting reliance on manual end of month stock reconciliation.

The developed system successfully integrates XGBoost-based demand forecasting with Mixed-Integer Linear Programming optimization to produce monthly replenishment recommendations for over 200 inventory items. The pipeline executes reliably within approximately 45 seconds, produces a structured and actionable Excel report and is accessible to non-technical users through a graphical interface.

**Key Accomplishments**

- A functioning predictive-prescriptive inventory decision system was developed and validated against real historical data.
- XGBoost models were trained for 207 of 221 items, with the remaining items handled through a mean-based fallback, ensuring complete coverage.
- The MILP optimization model correctly identifies items requiring replenishment and computes order quantities that minimize total inventory cost under capacity and safety stock constraints.
- The system reduces the manual effort required for inventory planning by providing a structured, automatically generated reorder report in place of end of month reconciliation.
- The graphical interface enables use by non-technical staff without programming knowledge.

**Limitations**

- The system is trained on 13 months of historical data. A longer history would improve forecast accuracy, particularly for items with seasonal demand patterns spanning multiple years.
- Cost parameters including holding cost rate, ordering cost, and stock out penalty were assigned standard assumptions rather than extracted from the company's financial records. Actual cost parameters would produce more accurate optimization outcomes.
- The system currently operates as a standalone desktop application and does not integrate with the company's point of sale system. Demand data must be exported manually to Excel before each run.
- The MILP model uses a single-period planning horizon. A multi-period formulation would account for lead time staggering and produce more sophisticated ordering schedules.

## 4.7 Recommendations

The following recommendations are proposed for future development based on the conclusions drawn from this study.

- Integrate the system with the company's point of sale platform to enable automatic data retrieval, eliminating the manual file export step and allowing the model to operate on real-time demand data.

- Extend the planning horizon of the MILP model from a single period to a multi-period formulation to enable more sophisticated ordering schedules that account for lead time variability and demand seasonality.
- Incorporate actual cost parameters by working with the company's financial records to obtain precise holding costs, ordering costs and stock out penalties. This would increase the accuracy and credibility of the optimization outcomes.
- Evaluate alternative forecasting models such as LSTM neural networks or Prophet for items exhibiting strong seasonal or trend-driven demand, and compare their performance against XGBoost using extended historical data.
- Implement automated retraining of the forecasting model on a monthly basis as new inventory data is recorded, ensuring that predictions remain current and reflective of evolving demand patterns.

## References


Pinto, L. M., & Ferreira, R. J. (2021). Mixed-Integer Linear Programming for inventory and production planning. International Journal of Production Economics.

Namazian, Z., Betts, J.M. & Stuckey, P.J. (2025). Predict and optimize: a smart inventory management model for beauty retail. Optim Lett

Munyoka, W., & Doyo, N. (2022). Inventory practices and performance in small enterprises in Africa. African Journal of Economic and Management Studies.

Nguyen, T. T. H., & Shiraki, Y. (2021). Inventory policies in SMEs under demand variability: A case analysis. Journal of Small Business Management.

Pinto, L. M., & Ferreira, R. J. (2021). Mixed-Integer Linear Programming for inventory and production planning. International Journal of Production Economic

Yousefi, S., & Lari, A. (2022). Simulation-optimization frameworks for inventory control. Simulation Modelling Practice and Theory.

Zhang, X., & Huang, Y. (2020). Inventory optimization under demand uncertainty: A review of methods. European Journal of Operational Research. `

Sani, A., Bertsimas, D., & Dunn, J. (2021). Predict to prescribe: An integrated decision-making framework. INFORMS Journal on Computing

Kumar, V., & Goyal, S.(2025). AI-Driven Forecasting and Optimization for Inventory

Control in Manufacturing Supply Chain. ACR Journal.

Vicente, J. J. (2025). Optimizing supply chain inventory: A mixed integer linear programming approach. Systems, 13(1), 33.

Bertsimas, D., & Kallus, N. (2018). From predictive to prescriptive analytics. arXis

Chopra, S., Meindl, P., & Kalra, D. V. (2019). Supply Chain Management: Strategy, Planning, and Operation (7th ed.). Pearson Education.

Essien, E. E., & Otu, U. O. E. (2024). An assessment of economic order quantity and organizational performance: A literature review. Business & Social Sciences Journal (BS SJ).

# APPENDICES

## APPENDIX A: PROJECT RESOURCES

This appendix outlines the key resources required to successfully conduct the research project.

### Hardware Resources

Personal computer or laptop.

Internet connectivity for literature access and software installation.

### Software Resources

Python Programming Language for data analysis, machine learning, and optimization

Python Libraries:

- Pandas – data pre-processing and analysis
- NumPy – numerical computations
- Scikit-learn – machine learning demand forecasting
- PuLP – Mixed-Integer Linear Programming modeling
- Matplotlib / Seaborn – visualization of results

VS Code – development environment

Microsoft Excel – data entry, cleaning, and preliminary analysis

Microsoft Word – documentation and report writing

### Data Resources

Historical sales data (quantities sold per period)

Inventory records (opening stock, closing stock, reorder quantities)

Cost parameters:

- Holding cost
- Ordering cost
- Stock-out or shortage cost

### Human Resources

Researcher (student)

Business owner or manager for operational insights and validation.

Academic supervisor (guidance and evaluation)

## APPENDIX B: PROJECT BUDGET

| Item | Description | Estimated Cost (KES) |
|---|---|---|
| Data Collection | Communication, printing, and transport | 5,000 |
| Internet Access | Literature access and software resources | 3,000 |
| Software Tools | Open-source tools (Python, PuLP, Scikit-learn) | 0 |
| Documentation | Printing and binding of final report | 2,500 |
| Contingency | Unforeseen project expenses | 1,500 |
| **Total Estimated Cost** | | **12,000** |

*Table 4: Budget*

## APPENDIX C: PROJECT SCHEDULE

| Weeks | Activity Description |
|---|---|
| Weeks 1–2 | System scoping, problem formalization, and data familiarization |
| Weeks 3–4 | Data pre-processing, exploratory analysis, and demand characterization |
| Weeks 5–6 | Development and validation of demand forecasting model |
| Weeks 7–8 | Formulation and implementation of the MILP inventory optimization model |
| Weeks 9–10 | Integration of forecasting outputs with optimization model and simulation setup |
| Week 11 | Performance evaluation and comparison with manual heuristic policies |
| Week 12 | Results interpretation, documentation, and final report preparation |

*Table 5: Schedule*

## APPENDIX D: GANTT CHART

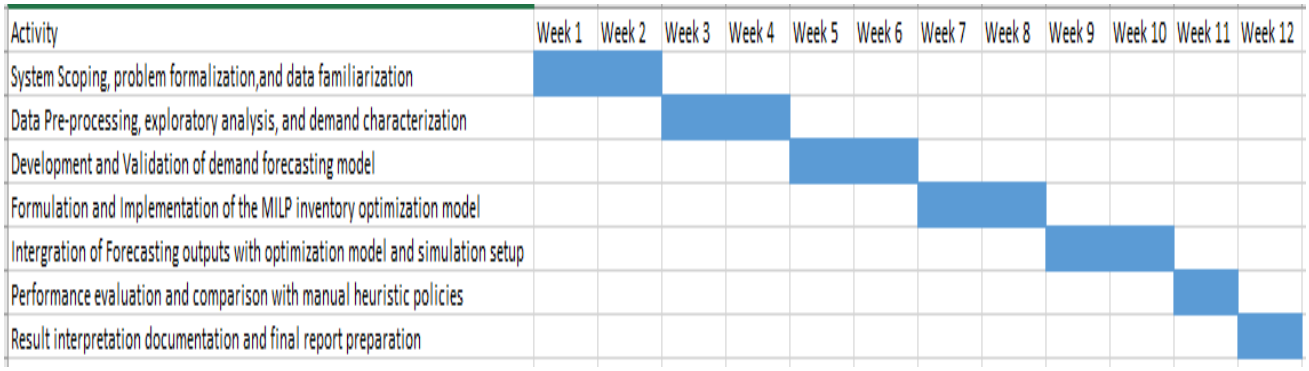


*FIGURE 10: Gantt Chart*

## APPENDIX E: INTERVIEW QUESTION

How do you currently track inventory levels?

How do you currently record inventory levels?

How do you decide when to place new orders?

How do you decide how much to order?

Do you experience stock shortages? How often do stock-outs occur?

Do you sometimes hold excess inventory?

What historical sales data is available?

How frequently is sales data recorded?

Do you use any forecasting methods currently?

How often do you review stock levels?

Would automated demand forecasting improve decision making?

What type of output would be most useful for order planning?